

Valutazione di OpenTelemetry per il monitoraggio di workload mixed-criticality

Campagna sperimentale su Linux PREEMPT-RT

Benito Cervone (secondo componente del gruppo)

28 agosto 2026

Questo documento riporta il lavoro svolto per valutare se e in che misura OpenTelemetry sia in grado di monitorare workload real-time a criticità mista, con particolare attenzione a due domande: se la pipeline di telemetria sappia prioritizzare i task critici rispetto a quelli best-effort, e quale sia l'overhead del monitoraggio sui tempi di risposta. La valutazione si basa su una campagna di 485 esecuzioni misurate su una piattaforma resa deterministica in modo controllato. Il codice strumentato è stato fornito come base di partenza; il nostro contributo riguarda la costruzione della piattaforma di misura, il disegno degli esperimenti, la loro esecuzione e l'analisi dei risultati.

Indice

1	La piattaforma sperimentale	3
1.1	Hardware e sistema	3
1.2	Parametri di avvio del kernel	3
1.3	Isolamento a runtime	4
1.4	Stabilità della frequenza	4
1.5	Costante di calibrazione	5
2	Il generatore di carico e le opzioni di strumentazione	5
2.1	Come si descrive un workload	5
2.2	Le opzioni che governano la strumentazione	6
3	Disegno degli esperimenti	7
3.1	Esperimento 1 — costo puro della strumentazione	7
3.2	Esperimento 2 — il campionamento protegge il task critico?	8
3.3	Esperimento 3 — consegna e contesa sotto carico crescente	8
3.4	Scelte metodologiche comuni	8
4	Esecuzione degli esperimenti	9
4.1	Preparazione della piattaforma	9
4.2	Definizione del workload	9
4.3	Definizione del tipo di monitoraggio	10
4.4	Esecuzione di una singola prova e di una campagna	10
4.5	Analisi	10

5	Analisi dei risultati	11
5.1	Esperimento 1: la scelta della variabile di risposta	11
5.2	Esperimento 2: il campionamento non distingue le criticità	12
5.3	Esperimento 3: la modalità di consegna determina il costo	13
5.4	Sintesi dei risultati	16
5.5	Limiti dello studio	16
6	Proposta: un criterio di campionamento consapevole della criticità	17
6.1	Il vincolo da cui partire	17
6.2	Due interventi complementari	17
6.3	Dove intervenire	18
6.4	Come verificarne l'efficacia	18
6.5	Osservazione conclusiva	18

1 La piattaforma sperimentale

Misurare l'overhead di una libreria di tracing significa cercare differenze dell'ordine delle decine di microsecondi. Su una macchina non preparata, la varianza dovuta a frequenza variabile, migrazioni di processo e interruzioni è di gran lunga superiore al segnale che si vuole osservare. La prima parte del lavoro è stata quindi dedicata a rendere la piattaforma sufficientemente stabile da rendere significative le misure successive.

1.1 Hardware e sistema

Le misure sono state eseguite su un portatile con processore AMD Ryzen 7 3700U (4 core fisici, 8 thread SMT, TDP 15 W) e kernel Linux 6.12.79-rt17 con patch PREEMPT-RT, installato in dual boot su hardware reale e non in virtualizzazione.

Il processore espone 8 CPU logiche appaiate a due a due sui 4 core fisici:

```
cpu0, cpu1 → core 0   cpu2, cpu3 → core 1   cpu4, cpu5 → core 2   cpu6, cpu7 → core
                                     3
```

Tutti i core condividono la stessa cache L3, per cui la scelta di *quali* core utilizzare è indifferente; conta invece che si tratti di core **interi**. Questo ha determinato due decisioni.

Il task critico e i task di disturbo devono stare su core fisici diversi. Due thread SMT dello stesso core condividono unità di esecuzione, cache L1 e L2: collocandoli lì, l'interferenza misurata sarebbe stata in buona parte contesa hardware, cioè esattamente il fenomeno che lo studio deve tenere separato dagli effetti di scheduling e di telemetria. Il task critico è stato quindi assegnato a `cpu2` (core 1) e i task di disturbo a `cpu6` (core 3).

Non si isola una CPU lasciando fuori il suo gemello SMT. Isolando ad esempio le sole `cpu2` e `cpu4`, il carico di sistema che continua a girare sui rispettivi gemelli sottrarrebbe risorse alle CPU nominalmente isolate, che risulterebbero tali solo sulla carta. L'isolamento comprende quindi i due core interi: 2,3,6,7.

1.2 Parametri di avvio del kernel

La riga di comando del kernel è stata configurata come segue:

```
isolcpus=managed_irq,domain,2,3,6,7 nohz_full=2,3,6,7
rcu_nocbs=2,3,6,7 irqaffinity=0,1,4,5
```

Ciascun parametro risponde a una sorgente di disturbo distinta.

- `isolcpus=domain` rimuove le CPU indicate dai domini di bilanciamento dello scheduler: nessun processo vi viene migrato automaticamente, e vi finisce solo ciò che viene collocato esplicitamente.
- `isolcpus=managed_irq` agisce sul lato di sottomissione delle richieste di I/O: il livello a code multiple del sottosistema a blocchi non utilizza le code il cui interrupt è associato a una CPU isolata. È utile chiarire un equivoco: questo parametro **non** modifica l'affinità degli interrupt gestiti dal kernel, che rimane invariata; l'effetto è che quelle code non vengono più utilizzate, e la verifica corretta è quindi il conteggio degli interrupt, non l'affinità dichiarata. Sulla nostra macchina le code del disco che ricadrebbero sulle CPU isolate mostrano contatori a zero.

- `nohz_full` elimina il tick periodico di scheduling sulle CPU indicate quando vi gira un solo task eseguibile. L'effetto è stato misurato e non assunto: in 5 secondi di esecuzione all'interno dell'area isolata si contano **106** interruzioni di temporizzazione locale sulla CPU del task critico, contro le 5000 che si avrebbero con il tick pieno alla frequenza configurata di 1000 Hz, e contro le 623–1216 osservate sulle CPU di servizio.
- `rcu_nocbs` sposta l'esecuzione dei callback di RCU fuori dalle CPU isolate.
- `irqaffinity` fa sì che gli interrupt *nascano* affini alle sole CPU di servizio. Senza questo parametro, alcune periferiche (controller grafico, USB, audio, tastiera) allocano i propri interrupt sulle CPU isolate, e occorre riassegnarli a posteriori. Dopo la verifica, tali interrupt risultano affini a 0-1, 4-5 e presentano conteggio nullo sulle CPU isolate già prima di attivare l'isolamento a runtime.

Restano due categorie non eliminabili, di cui si è tenuto conto: i thread di kernel per-CPU (migrazione, softirq, timer, RCU), che esistono per costruzione su ogni CPU e di cui l'isolamento riduce l'attività ma non l'esistenza; e un piccolo numero di interrupt gestiti direttamente dal kernel, la cui affinità non è modificabile ma che, come verificato, non vengono sollecitati.

1.3 Isolamento a runtime

Ai parametri di avvio si affianca un isolamento applicato a ogni sessione di misura, che crea due insiemi di CPU distinti: uno di servizio (0,1,4,5) in cui vengono confinati tutti i processi esistenti, e uno riservato (2,3,6,7) in cui gira soltanto ciò che viene lanciato esplicitamente. Lo stesso passo riassegna alle CPU di servizio le affinità degli interrupt che un driver avesse eventualmente allocato altrove dopo l'avvio.

L'efficacia è stata verificata sperimentalmente confrontando lo stesso carico dentro e fuori l'area isolata. Il jitter del periodo, definito come differenza fra periodo massimo e mediano, passa da 1420–2418 μ s all'esterno a 51–180 μ s all'interno, con un miglioramento di circa un ordine di grandezza. Sottoponendo la macchina a carico artificiale sulle CPU di servizio, i valori misurati all'interno dell'area isolata restano indistinguibili da quelli a macchina scarica, mentre all'esterno il periodo mediano cresce del 40 % con un terzo delle iterazioni perse.

1.4 Stabilità della frequenza

Il kernel real-time in uso è compilato senza il sottosistema di gestione della frequenza e senza quello degli stati di riposo del processore. Non esistono quindi i consueti governor, e gli strumenti abituali non sono applicabili: le transizioni di frequenza sono decise dal firmware e il sistema operativo può soltanto osservarle. Il rovescio positivo è che, non essendovi stati di riposo gestiti dal kernel, non si osservano picchi di latenza al risveglio; inoltre il contatore temporale del processore è indipendente dalla frequenza, per cui la misura del tempo resta corretta.

L'unica leva disponibile agisce sui registri di controllo del processore, ed è stata utilizzata per disabilitare il meccanismo di sovra-frequenza automatica e per richiedere lo stato di prestazione massimo. L'effetto è stato quantificato:

La dispersione del 31 % sulla frequenza si trasferisce direttamente sulle misure temporali, e sarebbe stata di gran lunga superiore all'overhead da misurare. Con la frequenza fissata, il valore osservato su tutte le 485 esecuzioni della campagna è risultato costante a **2295 MHz**, con temperatura compresa fra 48 e 57 °C e nessun intervento di limitazione termica.

	sovra-frequenza attiva	frequenza fissata
frequenza osservata sui core	2032–2670 MHz (dispersione 31 %)	2229–2296 MHz (2.9 %)
costo per iterazione stimato	18–21 ns (dispersione 17 %)	29–30 ns (3.4 %)
durata della fase di lavoro	+29/+44 % sul valore richiesto	+6/+9 %
periodo, 99° percentile	11572–12305 μ s	11165–11379 μ s

Tabella 1: Effetto della fissazione della frequenza, misurato su esecuzioni ripetute.

Le scritture su tali registri non sopravvivono al riavvio: vanno ripetute a ogni sessione, e il loro esito viene verificato automaticamente prima di ogni campagna, che si interrompe se la condizione non è soddisfatta.

1.5 Costante di calibrazione

Il generatore di carico determina la quantità di lavoro da eseguire dividendo la durata richiesta per un costo unitario stimato. Tale costo può essere misurato all'avvio oppure fissato nella configurazione. Abbiamo scelto di fissarlo, per due ragioni.

La prima è di determinismo: la misura automatica esegue attese ripetute fino a convergenza e richiede alcuni secondi non deterministici a ogni esecuzione, ma soprattutto restituisce un valore che dipende dallo stato istantaneo della macchina. Un valore diverso dell'1 % sposta la quantità di lavoro effettivamente eseguito del 3.5 %, cioè molto più dell'effetto che la campagna deve misurare.

La seconda è di comparabilità: con la costante fissata, tutte le celle dell'esperimento eseguono esattamente lo stesso numero di iterazioni del ciclo di carico, e le differenze osservate sono attribuibili al solo fattore in studio. Due campagne indipendenti hanno stimato il costo reale in circa 28.8 ns per iterazione; poiché il parametro accetta solo valori interi, la scelta ricade fra 28 (che darebbe un errore del +2.7 % sulla durata nominale) e 29 (−0.8 %). È stato adottato il valore **29** su tutte le esecuzioni.

2 Il generatore di carico e le opzioni di strumentazione

Il workload è generato da rt-app, un programma che esegue task periodici descritti in un file di configurazione. La versione utilizzata è stata fornita come base di partenza, già strumentata con OpenTelemetry, e ne abbiamo studiato il funzionamento per poterla configurare correttamente.

2.1 Come si descrive un workload

La configurazione è un file JSON con due sezioni. La sezione globale stabilisce la durata dell'esecuzione, la costante di calibrazione e la destinazione dei log; la sezione dei task descrive, per ciascun task, la politica di scheduling, la priorità, le CPU su cui può girare e la sequenza di eventi che ne costituisce la fase periodica.

Listing 1: Struttura della configurazione: un task critico e un task di disturbo.

```
{
  "global": {
    "duration": 20,           // durata dell'esecuzione, in secondi
    "calibration": 29,       // costo per iterazione, fissato (vedi 1.5)
    "logdir": "...",        // cartella dei log, una per esecuzione
    "log_basename": "rtapp"
```

```

},
"tasks": {
  "HI_task": {
    "policy": "SCHED_FIFO", // task critico
    "priority": 90,
    "cpus": [2], // core fisico 1
    "loop": -1, // ripete fino a fine durata
    "run": 2000, // 2000 us di lavoro effettivo
    "timer": { "ref": "unique", "period": 10000, "mode": "absolute" }
  },
  "LO_noise": {
    "policy": "SCHED_OTHER", // task di disturbo
    "instance": 4, // repliche indipendenti
    "cpus": [6], // core fisico 3
    "loop": -1,
    "run": 1000,
    "sleep": 1000
  }
}
}

```

Gli eventi che compongono la fase sono di tre tipi: **run** esegue un ciclo di calcolo per la durata indicata; **sleep** sospende il task per un tempo relativo alla fine del lavoro; **timer** sospende il task fino a un istante calcolato su una griglia temporale.

Il task critico usa un timer su griglia assoluta, e questa è una scelta deliberata. Con un'attesa relativa, l'istante di attivazione successivo deriva dalla somma fra lavoro e attesa, e ne eredita quindi tutta la variabilità; inoltre, in questa modalità il programma non calcola il margine residuo rispetto alla scadenza, che è la grandezza su cui si fonda tutta l'analisi successiva. Il confronto diretto fra le due modalità, a parità di tutto il resto, è riportato in Tabella 2.

	iterazioni	periodo mediano	jitter	margine calcolato
attesa relativa	1981/2000	10062 μ s	37 μ s	mai
timer su griglia assoluta	2000/2000	9999 μs	10 μs	sì

Tabella 2: Effetto della modalità di attivazione sul task critico, a parità di carico.

I task di disturbo usano invece l'attesa relativa, anch'essa per scelta. Essi sono volutamente in sovraccarico, e con un'attivazione su griglia assoluta un task in ritardo non attenderebbe più: il disturbo degenererebbe in un ciclo di calcolo continuo anziché mantenere il ciclo di lavoro previsto. Sui task di disturbo, del resto, non si misurano scadenze.

2.2 Le opzioni che governano la strumentazione

Il comportamento della telemetria è determinato da cinque parametri fissati **al momento della compilazione**, non modificabili a esecuzione avviata. Ne consegue che ogni configurazione sperimentale corrisponde a un eseguibile distinto, prodotto passando i relativi valori al compilatore. La Tabella 3 ne riassume il significato.

La distinzione fra i tre ruoli è importante per interpretare i risultati: il *sampler* decide *se*, il *processor* decide *come* e *quando*, l'*exporter* decide *dove*. Come si vedrà, il costo per il task critico dipende quasi interamente dal secondo.

Sul quinto parametro va fatta una precisazione metodologica. Nella configurazione predefinita gli elementi vengono inviati a un servizio di raccolta esterno; in assenza di tale servizio l'invio

parametro	valori	significato
livello di tracciamento	0, 1, 2, 3	Quanto è fine la granularità: 0 disattiva; 1 registra l'esecuzione complessiva e un elemento per thread; 2 aggiunge un elemento per fase; 3 aggiunge un elemento per ogni iterazione del ciclo, cioè il volume massimo.
tipo di <i>processor</i>	Batch, Simple	<i>Come</i> gli elementi raccolti vengono consegnati. Il primo li accumula in una coda e li invia a intervalli da un thread dedicato; il secondo li invia immediatamente, nel thread stesso che li ha generati . È il parametro più rilevante fra quelli studiati.
tipo di <i>sampler</i>	sempre, per proporzione, mai	<i>Se</i> una traccia viene registrata. Nel caso intermedio la decisione è probabilistica.
proporzione di campionamento	0...1	La probabilità usata dal caso intermedio.
tipo di <i>exporter</i>	collector, uscita standard	<i>Dove</i> gli elementi vengono inviati.

Tabella 3: I parametri che governano la strumentazione e ciò che ciascuno controlla.

fallisce senza bloccare l'esecuzione, ma non lascia alcuna traccia contabile di quanti elementi siano stati effettivamente prodotti. Poiché uno degli esperimenti richiede proprio di contarli, abbiamo reso selezionabile una modalità alternativa che scrive gli elementi sull'uscita standard, separata dal log del generatore di carico che viene invece emesso sull'uscita di errore. Questa aggiunta è stata realizzata rispettando i parametri esistenti, in modo che le due modalità differiscano *soltanto* per la destinazione e restino confrontabili sotto ogni altro aspetto.

3 Disegno degli esperimenti

La campagna è articolata in tre esperimenti, ciascuno costruito per isolare un solo fattore tenendo fissi tutti gli altri. Complessivamente sono state eseguite 485 esecuzioni misurate, riportate in Tabella 4.

	fattore studiato	celle	esecuzioni	carico
Esperimento 1	granularità del tracciamento	4	80	solo task critico
Esperimento 2	criterio di campionamento	6	150	1 critico + 4 di disturbo
Esperimento 3	modalità di consegna × carico	12	180	0, 1, 4, 8 di disturbo
verifica	controllo di riproducibilità	3	75	1 critico + 4 di disturbo

Tabella 4: Struttura della campagna sperimentale.

3.1 Esperimento 1 — costo puro della strumentazione

Obiettivo. Quantificare quanto costa al task critico la sola presenza della strumentazione, in assenza di qualunque altro carico.

Costruzione. Un solo task periodico critico, nessun task di disturbo, campionamento sempre attivo e consegna differita. L'unico fattore che varia è la granularità del tracciamento sui suoi

quattro livelli, con 20 ripetizioni per livello. L'assenza di carico concorrente è voluta: serve a ottenere un valore di riferimento non contaminato da contesa, rispetto al quale interpretare gli esperimenti successivi.

3.2 Esperimento 2 — il campionamento protegge il task critico?

Obiettivo. È la domanda centrale dello studio. In un sistema a criticità mista ci si attende di poter conservare integralmente le tracce dei task critici riducendo quelle dei task best-effort. L'esperimento verifica se il criterio di campionamento proporzionale offerto dalla libreria consenta questa distinzione.

Costruzione. Carico misto realistico (un task critico e quattro di disturbo), granularità e modalità di consegna fissate, e sei livelli del solo criterio di campionamento: sempre attivo, mai attivo, e quattro proporzioni intermedie (0.1, 0.3, 0.5, 0.7), con 25 ripetizioni ciascuno. Le 25 ripetizioni sono necessarie perché l'esito è probabilistico e la grandezza di interesse è una frequenza, di cui occorre stimare l'incertezza.

Variabile osservata. A differenza degli altri due esperimenti, qui non interessa principalmente il tempo, ma *quanti* elementi di traccia vengono effettivamente emessi e a quale task appartengono. Per questo l'esperimento adotta la modalità di uscita che li rende contabili.

3.3 Esperimento 3 — consegna e contesa sotto carico crescente

Obiettivo. Verificare come il costo della telemetria si comporti al crescere del numero di task concorrenti, e soprattutto se la modalità con cui gli elementi vengono consegnati incida sulla capacità del task critico di rispettare le proprie scadenze.

Costruzione. Due fattori incrociati: la modalità di consegna (differita o immediata) e il numero di task di disturbo (0, 1, 4, 8), con 15 ripetizioni per combinazione. A questi si affianca, per ogni livello di carico, una condizione di controllo con tracciamento disattivato, che fornisce il riferimento rispetto al quale misurare il costo a parità di contesa. È l'esperimento più vicino a una situazione d'esercizio.

3.4 Scelte metodologiche comuni

Numero di ripetizioni. Da 15 a 25 per cella, in linea con quanto richiesto per garantire significatività statistica, e dimensionato sulla variabilità osservata nelle prove preliminari.

Ordine di esecuzione interlacciato. Le ripetizioni non sono eseguite raggruppate per cella, ma alternando le celle a ogni ripetizione. Eseguire tutte le ripetizioni di una configurazione e poi quelle della successiva avrebbe fatto sì che qualunque deriva lungo la campagna — termica o di altra natura — si sommasse a una sola configurazione, diventando indistinguibile dal fattore in studio. Con l'ordine interlacciato, un'eventuale deriva agisce su tutte le celle allo stesso modo.

Verifiche preliminari bloccanti. Prima di ogni campagna si controlla automaticamente che l'isolamento delle CPU sia attivo e che la frequenza sia effettivamente fissata; in caso contrario l'esecuzione non parte. La cautela nasce da un'esperienza concreta: in assenza di isolamento la procedura di lancio ricade silenziosamente su un'esecuzione non isolata, e un'intera campagna risulterebbe inutilizzabile senza alcuna segnalazione.

Isolamento dei dati per esecuzione. Ogni ripetizione scrive in una propria cartella, che contiene anche la configurazione effettivamente eseguita. Questo evita la sovrascrittura dei log fra esecuzioni successive e rende ciascun risultato verificabile a posteriori.

Registrazione delle condizioni ambientali. A ogni esecuzione vengono registrate la frequenza osservata e la temperatura prima e dopo, in modo che un’eventuale deriva della piattaforma sia rilevabile nei dati anziché restare implicita.

Trattamento del transitorio iniziale. Le prime iterazioni di ogni esecuzione presentano un margine negativo per costruzione: il task critico fissa l’istante di riferimento e attende poi che tutti gli altri thread siano pronti, cosicché alla ripartenza l’origine dei tempi risulta già arretrata. Il numero di iterazioni interessate **cresce con il numero di thread** (da 1 con il solo task critico a circa 10 con otto task di disturbo). Poiché il numero di task di disturbo è uno dei fattori dell’Esperimento 3, scartare un numero fisso di righe avrebbe introdotto un errore sistematico correlato proprio con il fattore in studio, che si sarebbe letto come “più carico, più scadenze mancate”. L’analisi scarta quindi le righe iniziali fino al primo margine non negativo, registrando quante ne sono state escluse; oltre quel punto, un margine negativo è una scadenza realmente mancata.

4 Esecuzione degli esperimenti

Questa sezione riporta la sequenza di comandi che porta dalla macchina appena avviata ai dati analizzabili. Gli script citati sono quelli sviluppati per la campagna.

4.1 Preparazione della piattaforma

Da ripetere a ogni sessione, poiché nessuna delle due impostazioni sopravvive al riavvio.

```
# 1. frequenza fissata allo stato di prestazione massimo
sudo ./scripts/utils_isolation/pin_cpu_freq.sh fix 0
sudo ./scripts/utils_isolation/pin_cpu_freq.sh status # verifica

# 2. isolamento dei due core fisici destinati agli esperimenti
sudo ./scripts/utils_isolation/isolate_cpus.sh 2,3,6,7

# al termine della sessione
sudo ./scripts/utils_isolation/reset_isolation.sh
```

4.2 Definizione del workload

La configurazione viene generata a partire dai parametri del carico, così che le celle differiscano soltanto per ciò che si intende variare:

```
python3 scripts/measurements/gen_config.py \
    --n-lo 4          '# numero di task di disturbo' \
    --duration 20      '# durata in secondi' \
    --hi-cpu 2 --lo-cpus 6 '# core fisici distinti' \
    --calibration 29 \
    --out 1-configs/cfg_n4.json
```

Lo script avverte se il task critico e quelli di disturbo dovessero ricadere sui due thread dello stesso core fisico, condizione da evitare per le ragioni esposte in §1.1.

4.3 Definizione del tipo di monitoraggio

Poiché i parametri della telemetria sono fissati in compilazione, ogni configurazione corrisponde a un eseguibile distinto:

```
# esempio: granularita' massima, consegna immediata, campionamento sempre attivo
make -C rt-app/src CPPFLAGS="-DRTAPP_TRACE_LEVEL=3 \
                             -DRTAPP_PROCESSOR_TYPE=1 \
                             -DRTAPP_SAMPLER_TYPE=0 \
                             -DRTAPP_SAMPLER_RATIO=0.0 \
                             -DRTAPP_EXPORTER_TYPE=0"

# esempio: campionamento proporzionale al 30%, elementi contabili sull'uscita standard
make -C rt-app/src CPPFLAGS="-DRTAPP_TRACE_LEVEL=2 \
                             -DRTAPP_PROCESSOR_TYPE=0 \
                             -DRTAPP_SAMPLER_TYPE=1 \
                             -DRTAPP_SAMPLER_RATIO=0.3 \
                             -DRTAPP_EXPORTER_TYPE=1"
```

Gli eseguibili prodotti vengono conservati in una cache indicizzata dalla combinazione di parametri, in modo che ogni cella dell'esperimento usi esattamente il binario corrispondente e che la compilazione non venga ripetuta inutilmente.

4.4 Esecuzione di una singola prova e di una campagna

```
# singola esecuzione, all'interno dell'area isolata
bash scripts/measurements/test.sh <cartella_di_uscita> <eseguibile> <configurazione>

# campagna completa: costruisce gli eseguibili, genera le configurazioni,
# esegue le ripetizioni in ordine interlacciato e registra le condizioni ambientali
./scripts/measurements/run_doe.sh block1      # granularita' del tracciamento
./scripts/measurements/run_doe.sh block2      # criterio di campionamento
./scripts/measurements/run_doe.sh block3      # modalita' di consegna e carico
```

Ogni esecuzione produce la configurazione effettivamente usata, un log per ciascun thread, e le uscite standard e di errore. Al termine, una riga per esecuzione viene aggiunta a una tabella riassuntiva con i livelli dei fattori e le condizioni ambientali registrate.

4.5 Analisi

```
# estrae le variabili di risposta e le unisce ai livelli dei fattori
python3 scripts/measurements/analyze_doe.py \
    --data-table 2-DoE/data_table.csv --out 2-DoE/results.csv

# statistiche descrittive e confronti fra configurazioni
python3 scripts/measurements/report_doe.py \
    --results 2-DoE/results.csv --out 2-DoE/REPORT.md

# figure
python3 scripts/measurements/plot_doe.py
```

5 Analisi dei risultati

L'analisi è organizzata per esperimento. Si comincia dal primo perché, oltre a fornire il costo di riferimento, stabilisce *quale grandezza* sia lecito usare come variabile di risposta: una questione preliminare che condiziona la lettura di tutto il resto.

5.1 Esperimento 1: la scelta della variabile di risposta

Il generatore di carico riporta per ogni iterazione la durata della fase di lavoro e il periodo osservato. Sarebbe naturale usare la prima come misura del costo della strumentazione. La Figura 1, riquadro in basso a sinistra, mostra perché ciò condurrebbe a una conclusione falsa: il livello 1 di tracciamento risulta **più veloce** del livello 0, che non esegue alcuna strumentazione. Poiché un overhead non può accelerare il codice, il risultato è un artefatto.

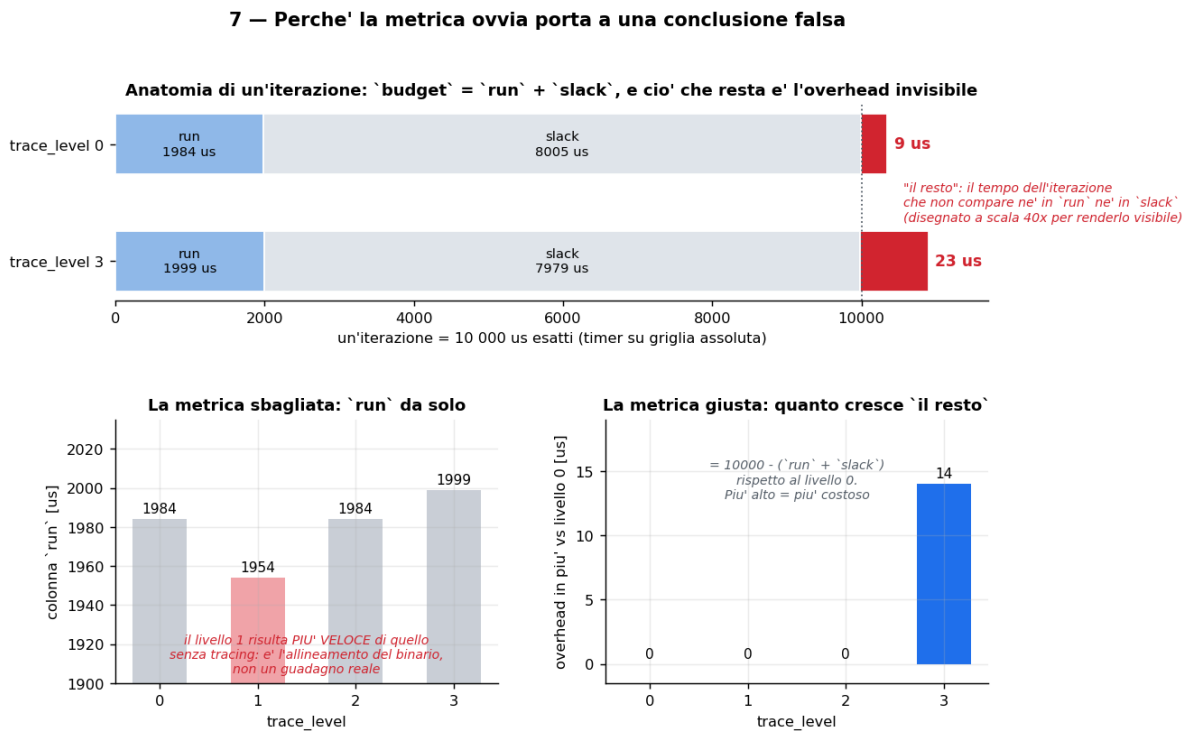


Figura 1: Anatomia di un'iterazione e confronto fra la grandezza riportata nativamente e quella effettivamente indicativa del costo.

La spiegazione richiede di guardare come si compone un'iterazione, riportata nel riquadro superiore. Poiché il task critico si attiva su una griglia temporale fissa, fra due attivazioni consecutive trascorrono esattamente 10 000 μ s. Tale intervallo si ripartisce in tre contributi:

- la **fase di lavoro**, l'unica cronometrata esplicitamente;
- il **marginale residuo** rispetto alla scadenza, misurato al termine del lavoro;
- la parte **non contabilizzata**, differenza fra il periodo e la somma delle due precedenti, che comprende la latenza di risveglio, la scrittura del log e — questo è il punto — la creazione e la chiusura degli elementi di traccia, che avvengono al di fuori della finestra cronometrata.

Definiamo *budget* la somma dei primi due contributi, cioè la porzione di iterazione che il generatore misura. L'overhead della telemetria non compare in essa, ma nella terza: quanto più la strumentazione lavora, tanto più il budget si riduce. L'ordinata del riquadro in basso a destra riporta perciò di quanto cresce la parte non contabilizzata rispetto alla condizione senza tracciamento.

livello	fase di lavoro	marginе residuo	budget	periodo	non contabilizzato
0	1984	8005	9991	10 000	9
1	1954	8036	9991	10 000	9
2	1984	8006	9991	10 000	9
3	1999	7979	9977	10 000	23

Tabella 5: Scomposizione dell'iterazione per livello di tracciamento; valori in microsecondi, mediane su 20 ripetizioni.

La riga del livello 1 chiarisce l'artefatto. La fase di lavoro *diminuisce* di $30\mu s$ rispetto al livello 0, ma il margine residuo *aumenta* di 31: i due si compensano e il budget resta invariato. Se quei $30\mu s$ fossero stati lavoro realmente risparmiato, l'iterazione sarebbe terminata prima e il margine sarebbe rimasto più ampio. Il tempo è semplicemente stato contabilizzato altrove: eseguibili diversi dispongono il codice in memoria in modo leggermente diverso, e la differenza di allineamento vale circa $30\mu s$, cioè **più del segnale da misurare**. Lo stesso vale per il periodo riportato, che al livello 3 risulta addirittura *più breve* proprio dove l'overhead cresce.

Risultato. Adottando il budget come variabile di risposta, i livelli 1 e 2 non producono alcun effetto misurabile, mentre il livello 3 costa **circa $14\mu s$ per iterazione**: lo 0.14 % del periodo, ma lo 0.7 % del lavoro utile. Non si registra alcuna scadenza mancata a nessun livello. La periodicità non viene degradata: l'intervallo fra attivazioni consecutive resta di $10\,000\mu s$ in tutte le condizioni.

È utile precisare che cosa *non* rientra in quei $14\mu s$. Scomponendo la parte non contabilizzata mediante la latenza di risveglio, che il generatore registra separatamente, si ottiene un contributo costante di circa $7.4\mu s$ dovuto al risveglio del thread e non attribuibile alla telemetria; i restanti $14\mu s$ rappresentano il costo di *costruire* gli elementi di traccia, non di trasmetterli. La trasmissione, in questa configurazione, è a carico di un thread separato: il suo costo emergerà nell'Esperimento 3.

5.2 Esperimento 2: il campionamento non distingue le criticità

È il risultato centrale dello studio.

Nella Figura 2 ogni punto rappresenta un'esecuzione. I punti si dispongono su due sole righe: nessun elemento emesso, oppure tutti e diciassette. **La banda intermedia è vuota in tutte e sei le configurazioni**: non esiste una singola esecuzione, su 150, in cui siano stati conservati gli elementi del task critico e scartati quelli dei task di disturbo, o viceversa.

Quando un'esecuzione viene campionata, gli elementi emessi sono sempre esattamente uno per il task critico e quattro per i task di disturbo. La ragione risiede nella struttura delle tracce: tutti i thread vengono creati come discendenti di un unico elemento radice che rappresenta l'esecuzione, e condividono di conseguenza il medesimo identificativo di traccia. Il criterio di campionamento proporzionale decide in funzione di tale identificativo, e la decisione viene quindi presa **una volta per esecuzione**, non per singolo elemento né per task.

La Figura 3 completa il quadro e va letta insieme alla precedente. La frazione di esecuzioni effettivamente campionate segue fedelmente la proporzione richiesta, e ogni intervallo di confi-

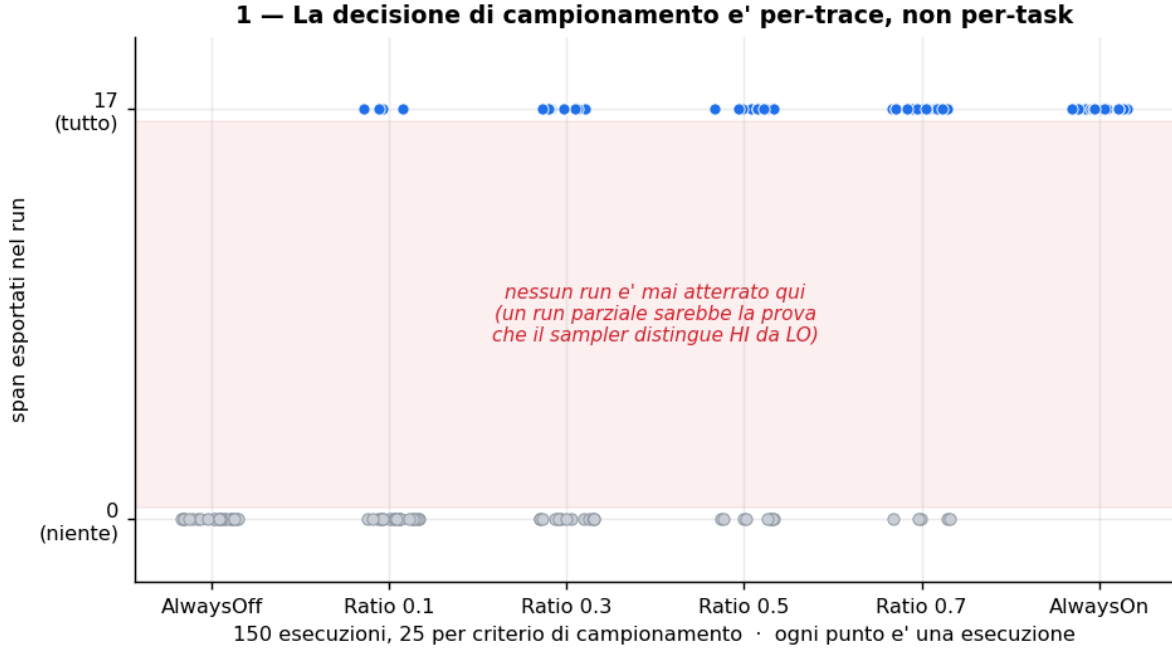


Figura 2: Elementi di traccia emessi in ciascuna delle 150 esecuzioni, per criterio di campionamento. La banda evidenziata copre tutti i valori intermedi.

denza contiene il valore nominale. Il meccanismo, dunque, **funziona correttamente**; ciò che non è adeguato è la granularità alla quale opera.

Conseguenza operativa. Impostare una proporzione di campionamento pari a 0.1 non significa “conserva il 10 % degli elementi”: significa “scarta l’intera esecuzione, task critico compreso, con probabilità del 90 %”. Nei nostri dati, a tale proporzione, in 21 esecuzioni su 25 **non esiste alcuna traccia del task critico**. In un sistema a criticità mista questo è il comportamento meno desiderabile fra quelli possibili: la riduzione del volume di telemetria, che è un’esigenza legittima, si ottiene sacrificando in modo indiscriminato proprio le informazioni che si vorrebbero preservare. È la constatazione da cui muove la proposta esposta nella §6.

Sul piano temporale, l’esperimento non evidenzia differenze: il budget per iterazione risulta pari a $9991\ \mu\text{s}$ in tutte e sei le configurazioni, campionamento disattivato compreso. Il confronto più pulito è interno alle configurazioni proporzionali, dove le esecuzioni si dividono in campionate e scartate in base al solo esito del sorteggio, a parità di eseguibile e configurazione: i due gruppi presentano lo stesso budget mediano, con differenza nulla entro la risoluzione della misura.

5.3 Esperimento 3: la modalità di consegna determina il costo

La Figura 4 riporta il costo per iterazione delle due modalità di consegna, misurato rispetto alla condizione di controllo allo stesso livello di carico. La consegna differita si mantiene stabile fra 12 e $14\ \mu\text{s}$ a ogni livello di carico, in accordo con il valore ottenuto nell’Esperimento 1. La consegna immediata costa circa $300\ \mu\text{s}$: ventitré volte tanto, pari al 3 % del periodo e al **15 % del lavoro utile**.

Il risultato sposta il baricentro delle conclusioni: non è la *quantità* di telemetria a determinare il costo, ma il *modo* in cui viene consegnata. A parità di granularità massima, e quindi di numero di elementi prodotti, il costo per il task critico varia di oltre un ordine di grandezza a seconda di questa sola scelta.

Un’avvertenza sulla lettura della figura. La barra relativa a quattro task di disturbo riporta un

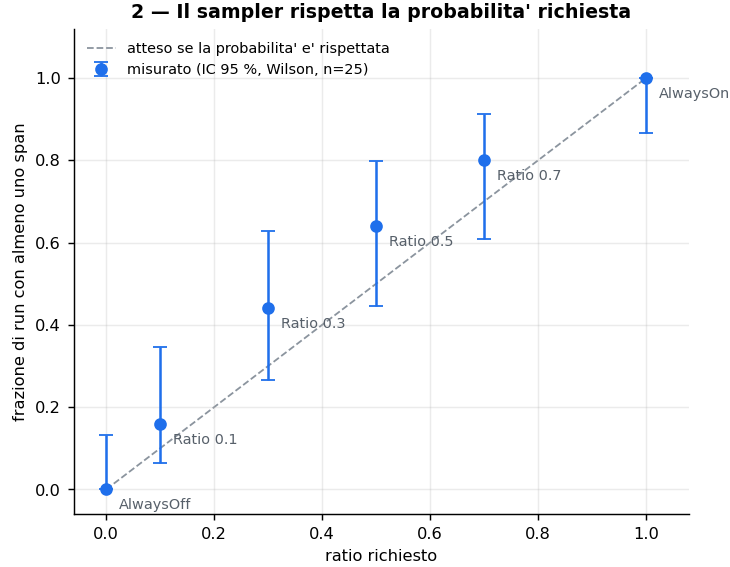


Figura 3: Frazione di esecuzioni campionate in funzione della proporzione richiesta; barre di incertezza al 95 % su 25 ripetizioni per configurazione.

valore di $1303 \mu s$, segnalato nella figura stessa: quella configurazione presenta un comportamento bimodale, con otto ripetizioni raggruppate attorno a un valore e sette attorno a un altro, e la mediana cade sul primo gruppo. Il valore non va quindi interpretato come un costo maggiore a quel livello di carico, e la non monotonia rispetto agli otto task di disturbo è solo apparente. Il costo attribuibile alla consegna immediata resta quello indicato di circa $300 \mu s$; l'origine del secondo comportamento non è stata determinata ed è segnalata fra i limiti dello studio.

La Figura 5 individua il meccanismo all'origine di tale divario, e non ne è una semplice riformulazione. Nella modalità immediata la trasmissione avviene **in modo sincrono, all'interno del thread che ha appena prodotto l'elemento**, sotto un blocco condiviso fra tutti i thread; nella modalità differita gli elementi vengono accodati e trasmessi da un thread dedicato a intervalli regolari. Il numero di tentativi di trasmissione per esecuzione passa da circa 230 a oltre 25 000: il task critico, nella prima modalità, sostiene di persona il costo di ogni singola trasmissione e concorre per il blocco condiviso con tutti gli altri thread.

Va sottolineato che nelle esecuzioni in esame **nessun servizio di raccolta era in ascolto**: i tentativi di trasmissione falliscono immediatamente. Questa è la condizione *più favorevole* alla modalità immediata; con un servizio realmente attivo, che accetta la connessione e risponde, il divario sarebbe maggiore. I valori riportati vanno quindi intesi come un limite inferiore.

La Figura 6 riporta la conseguenza sul rispetto delle scadenze. La condizione di controllo e quella a consegna differita risultano sovrapposte e **non raggiungono mai lo zero**: ogni iterazione si conclude con almeno $65 \mu s$ di margine. La consegna immediata presenta invece una coda che attraversa lo zero.

È l'unica figura che mostri il *margine* anziché il semplice conteggio dei fallimenti, e la distinzione è sostanziale. Nove scadenze mancate su circa 29 500 iterazioni (lo 0.03 %) potrebbero apparire trascurabili in forma tabellare; la distribuzione mostra invece che il comportamento è **qualitativamente diverso**, con una coda che si estende fino a $-3631 \mu s$: il task critico non sfiora occasionalmente di poco, ma può eccedere la propria scadenza di oltre un terzo del periodo. Si osserva inoltre che la separazione fra le due modalità è netta e priva di sovrapposizione, per cui la conclusione non dipende dalla scelta di una soglia.

Complessivamente, sulle 180 esecuzioni dell'esperimento le scadenze mancate sono 21, tutte nella

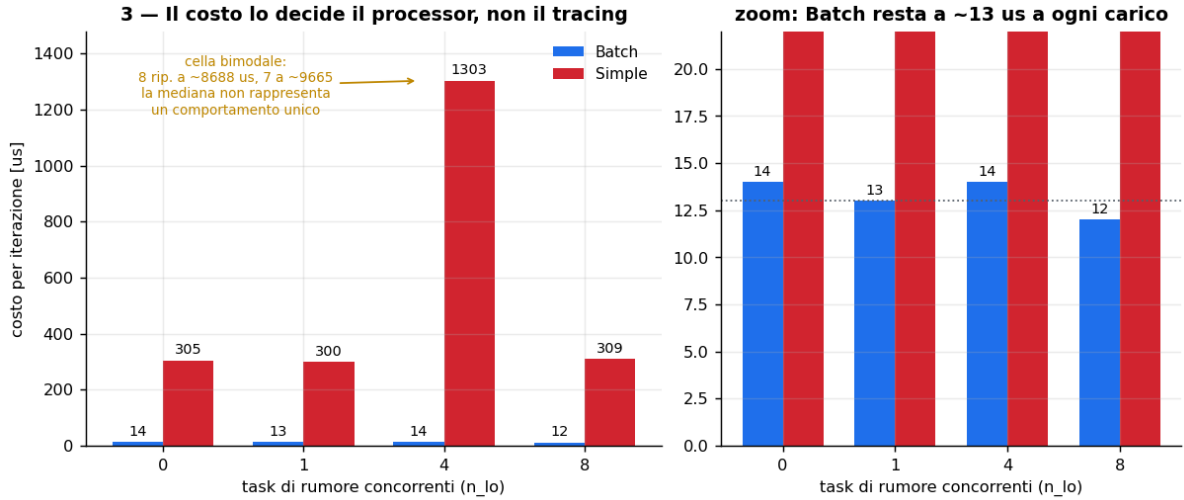


Figura 4: Costo per iterazione a carico crescente, rispetto alla condizione di controllo senza tracciamento. A destra la stessa grandezza con scala espansa.

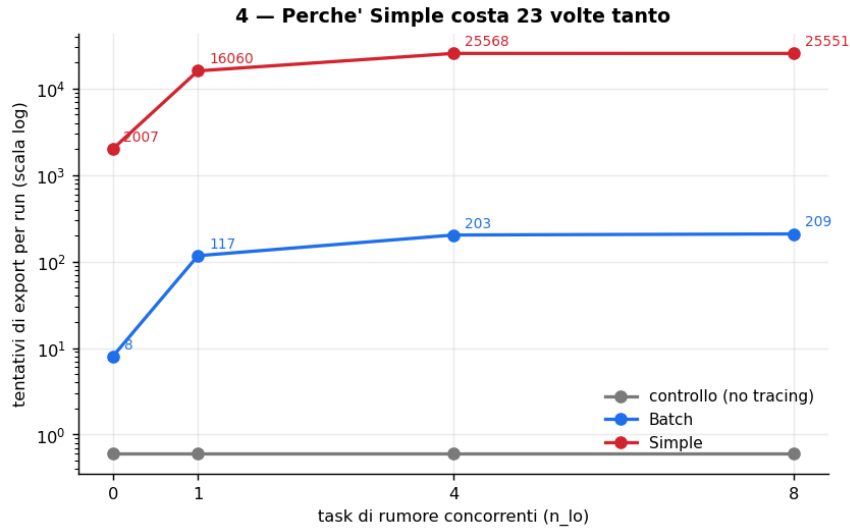


Figura 5: Numero di tentativi di trasmissione per esecuzione, in scala logaritmica.

modalità a consegna immediata e in presenza di carico concorrente; le condizioni di controllo e quelle a consegna differita ne registrano **zero a qualunque livello di carico**. Le scadenze mancate risultano distribuite uniformemente lungo l'esecuzione, e non concentrate in prossimità del termine: non sono quindi riconducibili alla fase di chiusura del programma.

Un'ulteriore osservazione riguarda la robustezza. Nella modalità a consegna immediata, in presenza di più thread, una frazione rilevante delle esecuzioni termina in modo anomalo al momento della chiusura, con probabilità crescente al crescere del numero di thread. Il fenomeno ha origine nell'interazione fra il modo in cui il generatore di carico termina i propri thread e il fatto che la funzione di consegna immediata sia dichiarata come non sollevante eccezioni: la terminazione attraversa tale funzione e provoca l'arresto del processo. L'anomalia si manifesta *dopo* che tutti i dati sono stati scritti, e non compromette quindi le misure; è però rilevante in prospettiva applicativa, poiché in questa configurazione la scelta della modalità di consegna non degrada soltanto le prestazioni del task critico, ma può determinarne la terminazione.

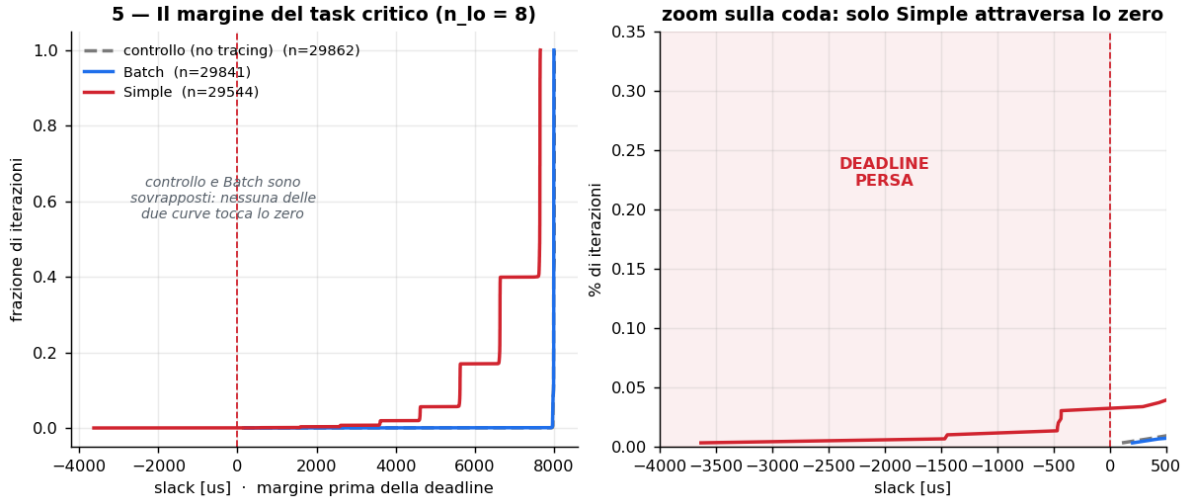


Figura 6: Distribuzione cumulativa del margine residuo del task critico al carico massimo; a destra l'ingrandimento della coda.

5.4 Sintesi dei risultati

domanda	risposta sperimentale
La pipeline di telemetria prioritizza i task critici?	No. La decisione di campionamento è presa per traccia e non per task: zero esecuzioni parziali su 150. Ridurre il volume significa perdere anche il task critico.
Quanto costa il monitoraggio al task critico?	Circa $14\mu s$ per iterazione con consegna differita, a qualunque livello di carico; circa $300\mu s$ con consegna immediata, pari al 15 % del lavoro utile.
Il monitoraggio compromette il rispetto delle scadenze?	Solo con consegna immediata e carico concorrente: 21 scadenze mancate, con margini fino a $-3631\mu s$. Con consegna differita, nessuna.
La granularità del tracciamento è il fattore determinante?	No: incide per $14\mu s$ soltanto al livello massimo. Il fattore determinante è la modalità di consegna.

Tabella 6: Sintesi delle evidenze raccolte.

5.5 Limiti dello studio

Assenza di un servizio di raccolta. Tutte le esecuzioni sono state condotte senza un servizio in ascolto. Come osservato, ciò rende i costi misurati un limite inferiore.

Configurazione con comportamento bimodale. Una delle dodici configurazioni dell'E-sperimento 3 presenta due comportamenti distinti fra le ripetizioni, per i quali non è stata individuata una spiegazione; il dato è stato riportato con l'avvertenza corrispondente anziché essere sintetizzato in un valore unico.

Piattaforma singola. Le misure provengono da un solo sistema, con quattro core fisici e un inviluppo termico limitato. I valori assoluti non sono trasferibili ad altro hardware; i confronti fra configurazioni, condotti a parità di ogni altra condizione, lo sono.

6 Proposta: un criterio di campionamento consapevole della criticità

I risultati dell'Esperimento 2 individuano un limite preciso e circoscritto: il criterio di campionamento proporzionale opera sull'identificativo di traccia, e poiché tutti i thread di un'esecuzione condividono tale identificativo, la decisione ricade sull'intera esecuzione. Non esiste, con i componenti disponibili, un modo di esprimere una politica del tipo “conserva sempre le tracce dei task critici e riduci quelle dei task best-effort”.

In questa sezione delineiamo come tale limite possa essere superato. L'esposizione resta a livello di progetto: si indicano i punti del sistema su cui intervenire e la logica da introdurre, senza scendere nel dettaglio implementativo.

6.1 Il vincolo da cui partire

L'analisi del funzionamento della libreria evidenzia due elementi che condizionano qualunque soluzione.

Il momento della decisione. Il componente che decide se registrare una traccia viene interpellato *all'atto della creazione* dell'elemento, e riceve il nome dell'elemento e i soli attributi passati contestualmente alla creazione. Nel codice a disposizione, gli attributi che descrivono la politica di scheduling e la priorità — cioè proprio le informazioni che identificano la criticità di un task — vengono impostati *successivamente*. Un criterio personalizzato, allo stato attuale, non li vedrebbe.

La struttura delle tracce. Ogni thread viene creato come discendente dell'elemento radice che rappresenta l'esecuzione. Questa scelta è ciò che rende l'intera esecuzione una traccia sola, ed è la causa diretta del comportamento osservato in Figura 2.

6.2 Due interventi complementari

Primo intervento: rendere la criticità visibile al momento della decisione. Sono possibili due strade, di costo diverso. La meno invasiva consiste nell'adottare una convenzione sui nomi dei task, che il criterio personalizzato può esaminare: il nome dell'elemento è infatti già disponibile al momento della decisione. Le configurazioni della campagna adottano già nomi distinti per i task critici e per quelli di disturbo, per cui questa strada non richiederebbe modifiche al codice esistente. La strada più pulita, ma che comporta una piccola modifica nel punto in cui gli elementi dei thread vengono creati, consiste nel passare la criticità come attributo contestualmente alla creazione, anziché subito dopo. È preferibile in prospettiva, perché non lega la politica a una convenzione sui nomi.

Secondo intervento: separare le tracce per criticità. È il punto essenziale, e senza di esso il primo non basta. Finché tutti i thread appartengono alla stessa traccia, qualunque criterio proporzionale continuerà a valutare un unico identificativo, e le decisioni sui diversi task resteranno rigidamente accoppiate. Occorre che *ciascun thread avvii una propria traccia*, così che ogni criticità possa essere campionata in modo indipendente.

La relazione causale con l'esecuzione principale, che ha valore diagnostico e non va perduta, può essere mantenuta sostituendo il legame di discendenza con un *collegamento* fra tracce, che la libreria prevede proprio per correlare tracce distinte. Il risultato è che le tracce restano navigabili come oggi, ma diventano campionabili separatamente.

6.3 Dove intervenire

Gli interventi si concentrano in tre punti, tutti circoscritti.

1. **Un nuovo componente di campionamento**, da introdurre come file a sé stante. Ne implementerebbe l'interfaccia prevista dalla libreria, esponendo una decisione basata sulla criticità riconosciuta: conservare sempre gli elementi dei task critici, applicare una proporzione configurabile agli altri, ed ereditare la decisione del genitore per gli elementi discendenti, così che una traccia campionata resti completa.
2. **La funzione di inizializzazione del tracciamento**, dove il fornitore di tracce viene costruito e dove viene scelto il criterio di campionamento. È il punto in cui il nuovo componente verrebbe registrato, affiancandolo a quelli esistenti secondo lo stesso schema già utilizzato per gli altri parametri di configurazione, in modo che la scelta resti selezionabile e non sostitutiva.
3. **Il punto di creazione degli elementi associati ai thread**, per i due aspetti discussi sopra: rendere disponibile la criticità al momento della decisione e avviare una traccia distinta per thread mantenendo il collegamento con l'esecuzione principale.

6.4 Come verificarne l'efficacia

La proposta è verificabile con la stessa impostazione sperimentale già costruita, il che ne rende la valutazione immediata. Sarebbe sufficiente ripetere l'Esperimento 2 sostituendo il criterio di campionamento, e osservare la Figura 2: con un criterio consapevole della criticità, i punti dovrebbero disporsi **nella banda intermedia**, oggi vuota, con gli elementi del task critico sempre presenti e quelli dei task di disturbo ridotti secondo la proporzione impostata. La stessa figura che oggi documenta il limite diventerebbe quindi la verifica del suo superamento.

Andrebbe inoltre misurato, con l'impostazione dell'Esperimento 1, il costo del nuovo criterio: la decisione viene presa a ogni creazione di elemento, e un criterio che esamini attributi o nomi è più oneroso di uno che valuti un solo identificativo. L'ordine di grandezza atteso è tale da non incidere in modo apprezzabile, ma trattandosi di un sistema real-time la verifica è necessaria e non può essere data per acquisita.

6.5 Osservazione conclusiva

Vale la pena rilevare che i due risultati principali di questo studio indicano due margini di intervento di natura diversa. Il costo di *costruire* gli elementi di traccia — circa $14\mu s$ per iterazione — è sostenuto necessariamente dal thread che li genera e non è eliminabile per via architetturale. Il costo di *consegnarli*, che come si è visto è di gran lunga superiore, è invece *collocabile*: la modalità differita lo sposta su un thread di servizio, sottraendolo al percorso critico. La proposta qui delineata agisce su un terzo asse, il *volume* della telemetria, rendendone la riduzione selettiva anziché indiscriminata.

Le tre leve sono indipendenti e componibili. Per un sistema a criticità mista, le evidenze raccolte suggeriscono di adottare la consegna differita, di non affidare al campionamento proporzionale la protezione dei task critici, e di introdurre un criterio in grado di distinguerli.